$lookup (aggregation stage)

Definition

`$lookup` Performs a left outer join to a collection in the *same* database to filter in documents from the foreign collection for processing. The `$lookup` stage adds a new array field to each input document. The new array field contains the matching documents from the foreign collection. The `$lookup` stage passes these reshaped documents to the next stage.

Starting in MongoDB 5.1, you can use `$lookup` with sharded collections.

To combine elements from two different collections, use the `$unionWith` pipeline stage.

Excessive use of `$lookup` may slow down query performance. To reduce reliance on `$lookup`, consider an embedded data model to store related data in a single collection.

For details on `$lookup` performance, see Performance Considerations.

Compatibility

`$lookup` You can use `$lookup` for deployments hosted in the following environments:

- MongoDB Atlas: The fully managed service for MongoDB deployments in the cloud
- MongoDB Enterprise: The subscription-based, self-managed version of MongoDB
- MongoDB Community: The source-available, free-to-use, and self-managed version of MongoDB

Syntax

The `$lookup` stage syntax:

```
{
   $lookup:
     {
       from: <collection to join>,
       localField: <field from the input documents>,
       foreignField: <field from the documents of the "from" collection>,
       let: { <var_1>: <expression>, …, <var_n>: <expression> },
       pipeline: [ <pipeline to run> ],
       as: <output array field>
     }
}
```

The `$lookup` accepts a document with these fields:

Field	Necessity	Description
from	Required	Specifies the foreign collection in the *same* database to join to the local collection. It is possible in some edge cases to subsitute `from` with `pipeline` with `$documents` as the first stage. For an example, see Use a $documents Stage in a $lookup Stage. Starting in MongoDB 5.1, the `from` collection can be sharded.
localField	Optional if `pipeline` is specified	Specifies the field from the documents input to the `$lookup` stage. `$lookup` performs an equality match on the `localField` to the `foreignField` from the documents of the `from` collection. If an input document does not contain the `localField`, the `$lookup` treats the field as having a value of `null` for matching purposes.
foreignField	Optional if `pipeline` is specified	Specifies the foreign documents' `foreignField` to perform an equality match with the local documents' `localField`. If a foreign document does not contain a `foreignField` value, the `$lookup` uses a `null` value for the match.
let	Optional	Specifies variables to use in the pipeline stages. Use the variable expressions to access the fields from the local collection's documents that are input to the `pipeline`. To reference variables in pipeline stages, use the `"$$"` syntax. The let variables can be accessed by the stages in the pipeline, including additional `$lookup` stages nested in the `pipeline`. - A `$match` stage requires the use of an `$expr` operator to access the variables. The `$expr` operator allows the use of aggregation expressions inside of the `$match` syntax. The `$eq`, `$lt`, `$lte`, `$gt`, and `$gte` comparison operators placed in an `$expr` operator can use an index on the `from` collection referenced in a `$lookup` stage. Limitations: - Indexes can only be used for comparisons between fields and constants, so the `let` operand must resolve to a constant. For example, a comparison between `$a` and a constant value can use an index, but a comparison between `$a` and `$b` cannot. - Indexes are not used for comparisons where the `let` operand resolves to an empty or missing value. - Multikey, partial, or sparse indexes are not used. -

		Other (non-`$match`) stages in the pipeline do not require an `$expr` operator to access the variables.
pipeline	Optional if `localField` and `foreignField` are specified	Specifies the `pipeline` to run on the foreign collection. The `pipeline` returns documents from the foreign collection. To return all documents, specify an empty `pipeline: []`. The `pipeline` cannot include the `$out` or `$merge` stages. Starting in v6.0, the `pipeline` can contain the MongoDB Search `$search` stage as the first stage inside the pipeline. To learn more, see MongoDB Search Support. The `pipeline` cannot access fields from input documents. Instead, define variables for the document fields using the let option and then reference the variables in the `pipeline` stages. To reference variables in pipeline stages, use the `"$$"` syntax. The let variables can be accessed by the stages in the pipeline, including additional `$lookup` stages nested in the `pipeline`. - A `$match` stage requires the use of an `$expr` operator to access the variables. The `$expr` operator allows the use of aggregation expressions inside of the `$match` syntax. The `$eq`, `$lt`, `$lte`, `$gt`, and `$gte` comparison operators placed in an `$expr` operator can use an index on the `from` collection referenced in a `$lookup` stage. Limitations: - Indexes can only be used for comparisons between fields and constants, so the `let` operand must resolve to a constant. For example, a comparison between `$a` and a constant value can use an index, but a comparison between `$a` and `$b` cannot. - Indexes are not used for comparisons where the `let` operand resolves to an empty or missing value. - Multikey, partial, or sparse indexes are not used. - Other (non-`$match`) stages in the pipeline do not require an `$expr` operator to access the variables.
as	Required	Specifies the name of the new array field to add to the input documents. The new array field contains the matching documents from the `from` collection. If the specified name already exists in the input document, the existing field is *overwritten*.

Equality Match with a Single Join Condition

To perform an equality match between a field from the input documents with a field from the documents of the foreign collection, the `$lookup` stage has this syntax:

```
{
   $lookup:
     {
       from: <collection to join>,
       localField: <field from the input documents>,
       foreignField: <field from the documents of the "from" collection>,
       pipeline: [ <pipeline to run> ],
       as: <output array field>
     }
}
```

In this example, `pipeline` is optional and runs after the local and foreign equality stage.

The operation corresponds to this pseudo-SQL statement:

```
SELECT *, (
   SELECT ARRAY_AGG(*)
   FROM <collection to join>
   WHERE <foreignField> = <collection.localField>
) AS <output array field>
FROM collection;
```

The SQL statements on this page are included for comparison to the MongoDB aggregation pipeline syntax. The SQL statements aren't runnable.

For MongoDB examples, see these pages:

- Perform a Single Equality Join with `$lookup`
- Use `$lookup` with an Array
- Use `$lookup` with `$mergeObjects`

Join Conditions and Subqueries on a Foreign Collection

MongoDB supports:

- Executing a pipeline on a foreign collection.

- Multiple join conditions.
- Correlated and uncorrelated subqueries.

In MongoDB, an uncorrelated subquery means that every input document will return the same result. A correlated subquery is a pipeline in a `$lookup` stage that uses the local or `input` collection's fields to return results correlated to each incoming document.

Starting in MongoDB 5.0, for an uncorrelated subquery in a `$lookup` pipeline stage containing a `$sample` stage, the `$sampleRate` operator, or the `$rand` operator, the subquery is always run again if repeated. Previously, depending on the subquery output size, either the subquery output was cached or the subquery was run again.

MongoDB correlated subqueries are comparable to SQL correlated subqueries, where the inner query references outer query values. An SQL uncorrelated subquery does not reference outer query values.

MongoDB 5.0 also supports concise correlated subqueries.

To perform correlated and uncorrelated subqueries with two collections, and perform other join conditions besides a single equality match, use this `$lookup` syntax:

```
{
   $lookup:
      {
         from: <foreign collection>,
         let: { <var_1>: <expression>, …, <var_n>: <expression> },
         pipeline: [ <pipeline to run on foreign collection> ],
         as: <output array field>
      }
}
```

The operation corresponds to this pseudo-SQL statement:

```
SELECT *, <output array field>
FROM collection
WHERE <output array field> IN (
   SELECT <documents as determined from the pipeline>
   FROM <collection to join>
   WHERE <pipeline>
);
```

See the following examples:

- Use Multiple Join Conditions and a Correlated Subquery
- Perform an Uncorrelated Subquery with `$lookup`

Correlated Subqueries Using Concise Syntax

Starting in MongoDB 5.0, you can use a concise syntax for a correlated subquery. Correlated subqueries reference document fields from a foreign collection *and* the "local" collection on which the `aggregate()` method was run.

The following new concise syntax removes the requirement for an equality match on the foreign and local fields inside of an `$expr` operator:

```
{
   $lookup:
      {
         from: <foreign collection>,
         localField: <field from local collection's documents>,
         foreignField: <field from foreign collection's documents>,
         let: { <var_1>: <expression>, …, <var_n>: <expression> },
         pipeline: [ <pipeline to run> ],
         as: <output array field>
      }
}
```

The operation corresponds to this pseudo-SQL statement:

```
SELECT *, <output array field>
FROM localCollection
WHERE <output array field> IN (
   SELECT <documents as determined from the pipeline>
   FROM <foreignCollection>
   WHERE <foreignCollection.foreignField> = <localCollection.localField>
   AND <pipeline match condition>
);
```

See this example:

- Perform a Concise Correlated Subquery with `$lookup`

Behavior

Encrypted Collections

Starting in MongoDB 8.1, you can reference multiple encrypted collections in a `$lookup` stage. However, `$lookup` does not support:

- Using an encrypted field as the join field in the `localField` or `foreignField`.

For drivers using Client-Side Field Level Encryption, you can use an encrypted field as a join field only if you are performing a self-join operation.

- Using any field in an encrypted array. An array is considered as encrypted if it contains any encrypted elements.
- For example, you can't use any field within the resulting as array of the `$lookup` operation, unless you're using Client-Side Field Level Encryption and `$unwind` the `as` field.

Views and Collation

If performing an aggregation that involves multiple views, such as with `$lookup` or `$graphLookup`, the views must have the same collation.

Restrictions

You cannot include the `$out` or the `$merge` stage in the `$lookup` stage. That is, when specifying a pipeline for the foreign collection, you cannot include either stage in the `pipeline` field.

```
{
   $lookup:
   {
      from: <collection to join>,
      let: { <var_1>: <expression>, …, <var_n>: <expression> },
      pipeline: [ <pipeline to execute on the foreign collection> ],  // Cannot
      as: <output array field>
   }
}
```

MongoDB Search Support

Starting in MongoDB 6.0, you can specify the MongoDB Search `$search` or `$searchMeta` stage in the `$lookup` pipeline to search collections on the Atlas cluster. The `$search` or the `$searchMeta` stage must be the first stage inside the `$lookup` pipeline.

For example, when you Join Conditions and Subqueries on a Foreign Collection or run Correlated Subqueries Using Concise Syntax, you can specify `$search` or `$searchMeta` inside the pipeline as shown below:

```
[{
  "$lookup": {
    "from": <foreign collection>,
    localField: <field from the input documents>,
    foreignField: <field from the documents of the "from" collection>,
    "as": <output array field>,
    "pipeline": [{
      "$search": {
        "<operator>": {
          <operator-specification>
        }
      },
      ...
    }]
  }
}]
```

```
[{
  "$lookup": {
    "from": <foreign collection>,
    localField: <field from the input documents>,
    foreignField: <field from the documents of the "from" collection>,
    "as": <output array field>,
    "pipeline": [{
      "$searchMeta": {
        "<collector>": {
          <collector-specification>
        }
      },
      ...
    }]
  }
}]
```

To see an example of `$lookup` with `$search`, see the MongoDB Search tutorial Run a MongoDB Search $search Query Using $lookup.

Sharded Collections

Starting in MongoDB 5.1, you can specify sharded collections in the `from` parameter of `$lookup` stages.

Starting in MongoDB 8.0, you can use the `$lookup` stage within a transaction while targeting a sharded collection.

Slot-Based Query Execution Engine

Starting in version 6.0, MongoDB can use the slot-based execution query engine to execute `$lookup` stages if *all* preceding stages in the pipeline can also be executed by the slot-based execution engine and none of the following conditions are true:

- The `$lookup` operation executes a pipeline on a foreign collection. To see an example of this kind of operation, see Join Conditions and Subqueries on a Foreign Collection.
- The `$lookup`'s `localField` or `foreignField` specify numeric components. For example: `{ localField: "restaurant.0.review" }`.
- The `from` field of any `$lookup` in the pipeline specifies a view or sharded collection.

For more information, see `$lookup` Optimization.

Performance Considerations

`$lookup` performance depends on the type of operation performed. Refer to the following table for performance considerations for different `$lookup` operations.

`$lookup` Operation	Performance Considerations
Equality Match with a Single Join	- `$lookup` operations that perform equality matches with a single join perform better when the foreign collection contains an index on the `foreignField`. IMPORTANT: If a supporting index on the `foreignField` does not exist, a `$lookup` operation that performs an equality match with a single join will likely have poor performance.
Uncorrelated Subqueries	- `$lookup` operations that contain uncorrelated subqueries perform better when the inner pipeline can reference an index of the foreign collection. - MongoDB only needs to run the `$lookup` subquery once before caching the query because there is no relationship between the source and foreign collections. The subquery is not based on any value in the source collection. This behavior improves performance for subsequent executions of the `$lookup` operation.
Correlated Subqueries	- `$lookup` operations that contain correlated subqueries perform better when the following conditions apply: - The foreign collection contains an index on the `foreignField`. - The foreign collection contains an index that references the inner pipline. - If your pipeline passes a large number of documents to the `$lookup` query, the following strategies may improve performance: - Reduce the number of documents that MongoDB passes to the `$lookup` query. For example, set a stricter filter during the `$match` stage. - Run the inner pipeline of the `$lookup` subquery as a separate query and use `$out` to create a temporary collection. Then, run an equality match with a single join. - Reconsider the data's schema to ensure it is optimal for the use case.

For general performance strategies, see Indexing Strategies and Query Optimization.

Examples

Perform a Single Equality Join with `$lookup`

Create a collection `orders` with these documents:

```
db.orders.insertMany( [
   { _id: 1, item: "almonds", price: 12, quantity: 2 },
   { _id: 2, item: "pecans", price: 20, quantity: 1 },
   { _id: 3  }
] )
```

Create another collection `inventory` with these documents:

```
db.inventory.insertMany( [
   { _id: 1, sku: "almonds", description: "product 1", instock: 120 },
   { _id: 2, sku: "bread", description: "product 2", instock: 80 },
   { _id: 3, sku: "cashews", description: "product 3", instock: 60 },
   { _id: 4, sku: "pecans", description: "product 4", instock: 70 },
   { _id: 5, sku: null, description: "Incomplete" },
   { _id: 6 }
] )
```

The following aggregation operation on the `orders` collection joins the documents from `orders` with the documents from the `inventory` collection using the fields `item` from the `orders` collection and the `sku` field from the `inventory` collection:

```
db.orders.aggregate( [
   {
     $lookup:
       {
         from: "inventory",
         localField: "item",
         foreignField: "sku",
         as: "inventory_docs"
       }
  }
] )
```

The operation returns these documents:

```
{
   _id: 1,
   item: "almonds",
   price: 12,
   quantity: 2,
   inventory_docs: [
      { _id: 1, sku: "almonds", description: "product 1", instock: 120 }
   ]
}
{
   _id: 2,
   item: "pecans",
   price: 20,
   quantity: 1,
   inventory_docs: [
      { _id: 4, sku: "pecans", description: "product 4", instock: 70 }
   ]
}
{
   _id: 3,
   inventory_docs: [
      { _id: 5, sku: null, description: "Incomplete" },
      { _id: 6 }
   ]
}
```

The operation corresponds to this pseudo-SQL statement:

```
SELECT *, inventory_docs
FROM orders
WHERE inventory_docs IN (
   SELECT *
   FROM inventory
   WHERE sku = orders.item
);
```

For more information, see Equality Match Performance Considerations.

Use `$lookup` with an Array

If the `localField` is an array, you can match the array elements against a scalar `foreignField` without an `$unwind` stage.

For example, create an example collection `classes` with these documents:

```
db.classes.insertMany( [
   { _id: 1, title: "Reading is ...", enrollmentlist: [ "giraffe2", "pandabear",
   { _id: 2, title: "But Writing ...", enrollmentlist: [ "giraffe1", "artie" ],
] )
```

Create another collection `members` with these documents:

```
db.members.insertMany( [
   { _id: 1, name: "artie", foreign: new Date("2016-05-01"), status: "A" },
   { _id: 2, name: "giraffe", foreign: new Date("2017-05-01"), status: "D" },
   { _id: 3, name: "giraffe1", foreign: new Date("2017-10-01"), status: "A" },
   { _id: 4, name: "panda", foreign: new Date("2018-10-11"), status: "A" },
   { _id: 5, name: "pandabear", foreign: new Date("2018-12-01"), status: "A" },
   { _id: 6, name: "giraffe2", foreign: new Date("2018-12-01"), status: "D" }
] )
```

The following aggregation operation joins documents in the `classes` collection with the `members` collection, matching on the `enrollmentlist` field to the `name` field:

```
db.classes.aggregate( [
   {
      $lookup:
         {
            from: "members",
            localField: "enrollmentlist",
            foreignField: "name",
            as: "enrollee_info"
         }
   }
] )
```

The operation returns the following:

```
{
   _id: 1,
   title: "Reading is ...",
   enrollmentlist: [ "giraffe2", "pandabear", "artie" ],
   days: [ "M", "W", "F" ],
   enrollee_info: [
      { _id: 1, name: "artie", foreign: ISODate("2016-05-01T00:00:00Z"), status:
      { _id: 5, name: "pandabear", foreign: ISODate("2018-12-01T00:00:00Z"), sta
      { _id: 6, name: "giraffe2", foreign: ISODate("2018-12-01T00:00:00Z"), stat
   ]
}
{
   _id: 2,
   title: "But Writing ...",
   enrollmentlist: [ "giraffe1", "artie" ],
   days: [ "T", "F" ],
   enrollee_info: [
      { _id: 1, name: "artie", foreign: ISODate("2016-05-01T00:00:00Z"), status:
      { _id: 3, name: "giraffe1", foreign: ISODate("2017-10-01T00:00:00Z"), stat
   ]
}
```

Use `$lookup` with `$mergeObjects`

The `$mergeObjects` operator combines multiple documents into a single document.

Create a collection `orders` with these documents:

```
db.orders.insertMany( [
   { _id: 1, item: "almonds", price: 12, quantity: 2 },
   { _id: 2, item: "pecans", price: 20, quantity: 1 }
] )
```

Create another collection `items` with these documents:

```
db.items.insertMany( [
  { _id: 1, item: "almonds", description: "almond clusters", instock: 120 },
  { _id: 2, item: "bread", description: "raisin and nut bread", instock: 80 },
  { _id: 3, item: "pecans", description: "candied pecans", instock: 60 }
] )
```

The following operation first uses the `$lookup` stage to join the two collections by the `item` fields and then uses `$mergeObjects` in the `$replaceRoot` to merge the foreign documents from `items` and `orders`:

```
db.orders.aggregate( [
   {
      $lookup: {
         from: "items",
         localField: "item",    // field in the orders collection
         foreignField: "item",  // field in the items collection
         as: "fromItems"
      }
   },
   {
      $replaceRoot: { newRoot: { $mergeObjects: [ { $arrayElemAt: [ "$fromItems"
   },
   { $project: { fromItems: 0 } }
] )
```

The operation returns these documents:

```
{
  _id: 1,
  item: 'almonds',
  description: 'almond clusters',
  instock: 120,
  price: 12,
  quantity: 2
},
{
  _id: 2,
  item: 'pecans',
  description: 'candied pecans',
  instock: 60,
  price: 20,
  quantity: 1
}
```

Use Multiple Join Conditions and a Correlated Subquery

Pipelines can execute on a foreign collection and include multiple join conditions. The `$expr` operator enables more complex join conditions including conjunctions and non-equality matches.

A join condition can reference a field in the local collection on which the `aggregate()` method was run and reference a field in the foreign collection. This allows a correlated subquery between the two collections.

MongoDB 5.0 supports concise correlated subqueries.

Create a collection `orders` with these documents:

```
db.orders.insertMany( [
  { _id: 1, item: "almonds", price: 12, ordered: 2 },
  { _id: 2, item: "pecans", price: 20, ordered: 1 },
  { _id: 3, item: "cookies", price: 10, ordered: 60 }
] )
```

Create another collection `warehouses` with these documents:

```
db.warehouses.insertMany( [
  { _id: 1, stock_item: "almonds", warehouse: "A", instock: 120 },
  { _id: 2, stock_item: "pecans", warehouse: "A", instock: 80 },
  { _id: 3, stock_item: "almonds", warehouse: "B", instock: 60 },
  { _id: 4, stock_item: "cookies", warehouse: "B", instock: 40 },
  { _id: 5, stock_item: "cookies", warehouse: "A", instock: 80 }
] )
```

The following example:

- Uses a correlated subquery with a join on the `orders.item` and `warehouse.stock_item` fields.
- Ensures the quantity of the item in stock can fulfill the ordered quantity.

```
db.orders.aggregate( [
   {
      $lookup:
         {
           from : "warehouses",
           localField : "item",
           foreignField : "stock_item",
           let : { order_qty: "$ordered" },
           pipeline : [
              { $match :
                 { $expr :
                      { $gte: [ "$instock", "$$order_qty" ] }
                 }
              },
              { $project : { stock_item: 0, _id: 0 } }
           ],
           as : "stockdata"
         }
    }
] )
```

The operation returns these documents:

```
{
  _id: 1,
  item: 'almonds',
  price: 12,
  ordered: 2,
  stockdata: [
    { warehouse: 'A', instock: 120 },
    { warehouse: 'B', instock: 60 }
  ]
},
{
  _id: 2,
  item: 'pecans',
  price: 20,
  ordered: 1,
  stockdata: [ { warehouse: 'A', instock: 80 } ]
},
{
  _id: 3,
  item: 'cookies',
  price: 10,
  ordered: 60,
  stockdata: [ { warehouse: 'A', instock: 80 } ]
}
```

The operation corresponds to this pseudo-SQL statement:

```
SELECT *, stockdata
FROM orders
WHERE stockdata IN (
   SELECT warehouse, instock
   FROM warehouses
   WHERE stock_item = orders.item
   AND instock >= orders.ordered
);
```

The `$eq`, `$lt`, `$lte`, `$gt`, and `$gte` comparison operators placed in an `$expr` operator can use an index on the `from` collection referenced in a `$lookup` stage. Limitations:

- Indexes can only be used for comparisons between fields and constants, so the `let` operand must resolve to a constant.

For example, a comparison between `$a` and a constant value can use an index, but a comparison between `$a` and `$b` cannot.

- Indexes are not used for comparisons where the `let` operand resolves to an empty or missing value.
- Multikey, partial, or sparse indexes are not used.

For example, if the index `{ stock_item: 1, instock: 1 }` exists on the `warehouses` collection:

- The equality match on the `warehouses.stock_item` field uses the index.
- The range part of the query on the `warehouses.instock` field also uses the indexed field in the compound index.
- `$expr`
- Variables in Aggregation Expressions

Perform an Uncorrelated Subquery with `$lookup`

An aggregation pipeline `$lookup` stage can execute a pipeline on the foreign collection, which allows uncorrelated subqueries. An uncorrelated subquery does not reference the local document fields.

Starting in MongoDB 5.0, for an uncorrelated subquery in a `$lookup` pipeline stage containing a `$sample` stage, the `$sampleRate` operator, or the `$rand` operator, the subquery is always run again if repeated. Previously, depending on the subquery output size, either the subquery output was cached or the subquery was run again.

Create a collection `absences` with these documents:

```
db.absences.insertMany( [
   { _id: 1, student: "Ann Aardvark", sickdays: [ new Date ("2018-05-01"),new Da
   { _id: 2, student: "Zoe Zebra", sickdays: [ new Date ("2018-02-01"),new Date
] )
```

Create another collection `holidays` with these documents:

```
db.holidays.insertMany( [
   { _id: 1, year: 2018, name: "New Years", date: new Date("2018-01-01") },
   { _id: 2, year: 2018, name: "Pi Day", date: new Date("2018-03-14") },
   { _id: 3, year: 2018, name: "Ice Cream Day", date: new Date("2018-07-15") },
   { _id: 4, year: 2017, name: "New Years", date: new Date("2017-01-01") },
   { _id: 5, year: 2017, name: "Ice Cream Day", date: new Date("2017-07-16") }
] )
```

The following operation joins the `absences` collection with 2018 holiday information from the `holidays` collection:

```
db.absences.aggregate( [
   {
      $lookup:
         {
           from: "holidays",
           pipeline: [
              { $match: { year: 2018 } },
              { $project: { _id: 0, date: { name: "$name", date: "$date" } } },
              { $replaceRoot: { newRoot: "$date" } }
           ],
           as: "holidays"
         }
    }
] )
```

The operation returns the following:

```
{
  _id: 1,
  student: 'Ann Aardvark',
  sickdays: [
    ISODate("2018-05-01T00:00:00.000Z"),
    ISODate("2018-08-23T00:00:00.000Z")
  ],
  holidays: [
    { name: 'New Years', date: ISODate("2018-01-01T00:00:00.000Z") },
    { name: 'Pi Day', date: ISODate("2018-03-14T00:00:00.000Z") },
    { name: 'Ice Cream Day', date: ISODate("2018-07-15T00:00:00.000Z")
    }
  ]
},
{
  _id: 2,
  student: 'Zoe Zebra',
  sickdays: [
    ISODate("2018-02-01T00:00:00.000Z"),
    ISODate("2018-05-23T00:00:00.000Z")
  ],
  holidays: [
    { name: 'New Years', date: ISODate("2018-01-01T00:00:00.000Z") },
    { name: 'Pi Day', date: ISODate("2018-03-14T00:00:00.000Z") },
    { name: 'Ice Cream Day', date: ISODate("2018-07-15T00:00:00.000Z")
    }
  ]
}
```

The operation corresponds to this pseudo-SQL statement:

```
SELECT *, holidays
FROM absences
WHERE holidays IN (
   SELECT name, date
   FROM holidays
   WHERE year = 2018
);
```

For more information, see Uncorrelated Subquery Performance Considerations.

Perform a Concise Correlated Subquery with `$lookup`

Starting in MongoDB 5.0, an aggregation pipeline `$lookup` stage supports a concise correlated subquery syntax that improves joins between collections. The new concise syntax removes the requirement for an equality match on the foreign and local fields inside of an `$expr` operator in a `$match` stage.

Create a collection `restaurants`:

```
db.restaurants.insertMany( [
   {
      _id: 1,
      name: "American Steak House",
      food: [ "filet", "sirloin" ],
      beverages: [ "beer", "wine" ]
   },
   {
      _id: 2,
      name: "Honest John Pizza",
      food: [ "cheese pizza", "pepperoni pizza" ],
      beverages: [ "soda" ]
   }
] )
```

Create another collection `orders` with food and optional drink orders:

```
db.orders.insertMany( [
   {
      _id: 1,
      item: "filet",
      restaurant_name: "American Steak House"
   },
   {
      _id: 2,
      item: "cheese pizza",
      restaurant_name: "Honest John Pizza",
      drink: "lemonade"
   },
   {
      _id: 3,
      item: "cheese pizza",
      restaurant_name: "Honest John Pizza",
      drink: "soda"
   }
] )
```

The following example:

- Joins the `orders` and `restaurants` collections by matching the `orders.restaurant_name` localField with the `restaurants.name` foreignField. The match is performed before the `pipeline` is run.
- Performs an `$in` array match between the `orders.drink` and `restaurants.beverages` fields that are accessed using `$$orders_drink` and `$beverages` respectively.

```
db.orders.aggregate( [
   {
      $lookup: {
         from: "restaurants",
         localField: "restaurant_name",
         foreignField: "name",
         let: { orders_drink: "$drink" },
         pipeline: [ {
            $match: {
               $expr: { $in: [ "$$orders_drink", "$beverages" ] }
            }
         } ],
         as: "matches"
      }
   }
] )
```

There is a match for the `soda` value in the `orders.drink` and `restaurants.beverages` fields. This output shows the `matches` array and contains all foreign fields from the `restaurants` collection for the match:

```
{
   _id: 1, item: "filet",
   restaurant_name: "American Steak House",
   matches: [ ]
}
{
   _id: 2, item: "cheese pizza",
   restaurant_name: "Honest John Pizza",
   drink: "lemonade",
   matches: [ ]
}
{
   _id: 3, item: "cheese pizza",
   restaurant_name: "Honest John Pizza",
   drink: "soda",
   matches: [ {
      _id: 2, name": "Honest John Pizza",
      food: [ "cheese pizza", "pepperoni pizza" ],
      beverages: [ "soda" ]
   } ]
}
```

This example uses the older verbose syntax from MongoDB versions before 5.0 and returns the same results as the previous concise example:

```
db.orders.aggregate( [
   {
      $lookup: {
         from: "restaurants",
         let: { orders_restaurant_name: "$restaurant_name",
                orders_drink: "$drink" },
         pipeline: [ {
            $match: {
               $expr: {
                  $and: [
                     { $eq: [ "$$orders_restaurant_name", "$name" ] },
                     { $in: [ "$$orders_drink", "$beverages" ] }
                  ]
               }
            }
         } ],
         as: "matches"
      }
   }
] )
```

The previous examples correspond to this pseudo-SQL statement:

```
SELECT *, matches
FROM orders
WHERE matches IN (
   SELECT *
   FROM restaurants
   WHERE restaurants.name = orders.restaurant_name
   AND restaurants.beverages = orders.drink
);
```

For more information, see Correlated Subquery Performance Considerations.

Namespaces in Subpipelines

Starting in MongoDB 8.0, namespaces in subpipelines within `$lookup` and `$unionWith` are validated to ensure the correct use of `from` and `coll` fields:

- For `$lookup`, omit the `from` field if you use a subpipeline with a stage which doesn't require a specified collection. For example, a `$documents` stage.

- Similarly, for `$unionWith`, omit the `coll` field.

Unchanged behavior:

- For a `$lookup` that starts with a stage for a collection, for example a `$match` or `$collStats` subpipeline, you must include the `from` field and specify the collection.
- Similarly, for `$unionWith`, include the `coll` field and specify the collection.

The following scenario shows an example.

Create a collection `cakeFlavors`:

```
db.cakeFlavors.insertMany( [
   { _id: 1, flavor: "chocolate" },
   { _id: 2, flavor: "strawberry" },
   { _id: 3, flavor: "cherry" }
] )
```

Starting in MongoDB 8.0, the following example returns an error because it contains an invalid `from` field:

```
db.cakeFlavors.aggregate( [ {
   $lookup: {
      from: "cakeFlavors",
      pipeline: [ { $documents: [ {} ] } ],
      as: "test"
   }
} ] )
```

In MongoDB versions before 8.0, the previous example runs.

For an example with a valid `from` field, see Perform a Single Equality Join with `$lookup`.

The C# examples on this page use the `sample_mflix` database from the Atlas sample datasets. To learn how to create a free MongoDB Atlas cluster and load the sample datasets, see Get Started in the MongoDB .NET/C# Driver documentation.

The following `Movie` class models the documents in the `sample_mflix.movies` collection:

```
public class Movie
{
    public ObjectId Id { get; set; }

    public int Runtime { get; set; }

    public string Title { get; set; }

    public string Rated { get; set; }

    public List<string> Genres { get; set; }

    public string Plot { get; set; }

    public ImdbData Imdb { get; set; }

    public int Year { get; set; }

    public int Index { get; set; }

    public string[] Comments { get; set; }

    [BsonElement("lastupdated")]
    public DateTime LastUpdated { get; set; }
}
```

The C# classes on this page use Pascal case for their property names, but the field names in the MongoDB collection use camel case. To account for this difference, you can use the following code to register a `ConventionPack` when your application starts:

```
var camelCaseConvention = new ConventionPack { new CamelCaseElementNameConventio
ConventionRegistry.Register("CamelCase", camelCaseConvention, type => true);
```

The following `Comment` class models the documents in the `sample_mflix.comments` collection:

```
public class Comment
{
    public Guid Id { get; set; }

    [BsonElement("movie_id")]
    public Guid MovieId { get; set; }

    public string Text { get; set; }
}
```

`$lookup`

Lookup()

performs a left outer join between the `movies` and `comments` collections. The code joins the `Id` field from each `Movie` document to the `MovieId` field in the `Comment` documents. The comments for each movie are stored in a field named `Comments` in each `Movie` document.

To use the MongoDB .NET/C# driver to add a `$lookup` stage to an aggregation pipeline, call the Lookup() method on a `PipelineDefinition` object.

The following example creates a pipeline stage that performs a left outer join between the `movies` and `comments` collections. The code joins the `Id` field from each `Movie` document to the `MovieId` field in the `Comment` documents. The comments for each movie are stored in a field named `Comments` in each `Movie` document.

```
var commentCollection = client
    .GetDatabase("aggregation_examples")
    .GetCollection<Comment>("comments");

var pipeline = new EmptyPipelineDefinition<Movie>()
    .Lookup<Movie, Movie, Comment, Movie>(
        foreignCollection: commentCollection,
        localField: m => m.Id,
        foreignField: c => c.MovieId,
        @as: m => m.Comments);
```

The Node.js examples on this page use the `sample_mflix` database from the Atlas sample datasets. To learn how to create a free MongoDB Atlas cluster and load the sample datasets, see Get Started in the MongoDB Node.js driver documentation.

`$lookup`

performs a left outer join between the `movies` and `comments` collections. The code joins the `_id` field from each `movie` document to the `movie_id` field in the `comment` documents. The `comments` field stores the comments for each movie in each `movie` document

To use the MongoDB Node.js driver to add a `$lookup` stage to an aggregation pipeline, use the `$lookup` operator in a pipeline object.

The following example creates a pipeline stage that performs a left outer join between the `movies` and `comments` collections. The code joins the `_id` field from each `movie` document to the `movie_id` field in the `comment` documents. The `comments` field stores the comments for each movie in each `movie` document. The example then runs the aggregation pipeline:

```
const pipeline = [
  {
    $lookup: {
      from: "comments",
      localField: "_id",
      foreignField: "movie_id",
      as: "comments"
    }
  }
];

const cursor = collection.aggregate(pipeline);
return cursor;
```